Highlights

Efficient Communication Between a Non-Transformer Perception Agent and a Frozen Gemma Language Model: A Communication-Fidelity Benchmark of Filtered Text and a Latent Channel

Alif Tasbir, Aboozar Taherkhani

- Latent channel from a non-transformer sender compared with filtered text
- Adapter carries up to 50 events at four tokens each into a frozen model
- Filtered text is more accurate; the adapter wins under batched serving
- Per-slot, per-field recoverability of the virtual tokens is measured
- A selection rule, the analysis plan and its change log are released

Efficient Communication Between a Non-Transformer Perception Agent and a Frozen Gemma Language Model: A Communication-Fidelity Benchmark of Filtered Text and a Latent Channel

Alif Tasbir[a,*], Aboozar Taherkhani[a]

[a]*De Montfort University, Leicester, United Kingdom*

Abstract

Messages between language-model agents cost time and energy, especially on edge hardware. Latent communication, which replaces text with continuous messages, has been studied only between transformers, yet sensing agents are often not transformers. The AI contribution is a learned linear adapter that lets a non-transformer sender speak to a frozen language model, and a benchmark to evaluate it; the application is heart-rhythm triage on edge hardware. A heartbeat classifier passes events to a frozen Gemma 4 E4B model as text, including a filtered message listing only abnormal events, or through the adapter, which emits four virtual tokens per event. The task, reporting the most urgent triage tier among 1 to 50 heartbeats, has a fixed rule as its answer, so errors measure the channel. With three seeds and held-out patients, the adapter cut prompt-processing time by 24%, 50% and 78% against compact text at 10, 20 and 50 events and was more accurate than calibrated text at 10 and 20 events (balanced accuracy 0.83 against 0.73). Filtered text was 0.11 to 0.17 more accurate and equally fast for single requests, but under batched serving on one 12 GB graphics card the adapter's median throughput was 10 to 30% higher and energy per decision 12 to 28% lower. Heart rate, interval and abnormal runs could be recovered from the tokens. The pattern largely held on a second sender and an external database. Filtering is preferable when the question is known in advance; a latent channel suits many events carried at predictable cost.

Keywords: agent communication, latent communication, communication efficiency, heterogeneous agents, edge inference, electrocardiogram

1. Introduction

Agents built on language models spend much of their budget talking to each other. The tokens exchanged grow with every hand-off, and much of that traffic is redundant [1]. The cost appears as latency and energy and, on edge hardware, where memory is fixed and the longest prompt sets the batch size, as a limit on how many requests can be served at all [2, 3]. Communication efficiency is now a design problem in its own right [4].

The literature offers three levers. The first is to *say less*: prune redundant messages from the agent graph [1], send only the semantic packets whose expected contribution exceeds their cost [5], or compress the prompt [6, 7, 8, 9]. The second is to *reuse computation*: persist and share caches so that a receiver does not re-read what it has already read [3]. The third is to *send a latent message* instead of text, handing the receiver hidden states, a key–value cache or an embedding with no decoding into words. The training-free LatentMAS reports 70.8 to 83.7% fewer tokens and 4 to 4.3 times faster

*Corresponding author

Email addresses: aliftasbir@gmail.com (Alif Tasbir), aboozar.taherkhani@dmu.ac.uk (Aboozar Taherkhani)

inference than text on nine benchmarks [10]; cache-to-cache reports a 2.5-fold reduction in latency and 3.1 to 5.4% higher accuracy than text [11]. Other designs follow the same pattern [12, 13, 14, 15, 16, 17], and a recent framework organises them by what is transmitted and how the receiver absorbs it [18]. Systems have begun to choose among these levers adaptively, for instance between token and cache transmission according to bandwidth [19].

All of this work assumes that the agents are language models, so that the sender's hidden state or cache is something another transformer can be trained to absorb. The agents that sense the world usually are not transformers. A wearable or bedside classifier is a convolutional or recurrent network trained for classification, and nothing about its internal vector is shaped for a language model. The one heterogeneous exception we know of connects different families of vision–language models through a shared visual channel [20], but its senders are still language models. For a sender of this kind it is open whether a latent channel is feasible, whether it is worth building, and what it costs in inspectability.

Neighbouring work does not settle the question. Projecting a signal encoder into a language model's embedding space is established [21, 22, 23, 24], but those systems are built to make a language model understand a signal and are judged on accuracy. They do not treat the projection as a channel whose cost is compared with a text channel on the same task. Comparisons of latent and text communication also tend to use the text baseline that is easiest to beat. Listing every event verbatim is a poor choice when the sender can filter, which is the first lever above. Without a filtered baseline, a saving in tokens says little about whether a latent channel is worth building.

This paper compares the levers for a non-transformer sender. A small learned adapter maps the 32-dimensional context vector of a convolutional–recurrent heartbeat classifier, together with three scaled side inputs, to four virtual tokens per event in the input-embedding space of a frozen, 4-bit Gemma 4 E4B model. We compare it with three text interfaces, among them a sender-side filter, each also measured with prefix caching. The task is to report the most urgent triage tier among 1 to 50 consecutive heartbeats.

Electrocardiography is the case study: deep networks already match cardiologists on it [25] and language models encode clinical knowledge [26], so pairing a sensing agent with a reasoning agent is a natural design. The task tests *communication fidelity*. The correct answer is a fixed rule applied to the sender's outputs, so a wrong answer is a failure of the channel and not of clinical knowledge. A three-line rule also solves the task, so the language model adds nothing to the decision here. It stands in for the general-purpose receiver of an agent system, a model shared by many senders and tasks for which no one writes a rule per task, and the benchmark isolates the channel into it. The experiment therefore measures what each interface costs and loses, under a protocol fixed before the results existed; the results tables report the rule itself as the ceiling. We ask three questions.

- Does the adapter reduce the tokens, time and memory of the hand-off relative to text interfaces, and under which workloads?
- Does it preserve the accuracy of the downstream decision?
- How much of each event's content can be recovered from the virtual tokens, so that the cost of opacity is measured and not asserted?

The answers do not all favour the latent channel. It keeps prompt processing flat as events are added, while text grows by about 60 tokens per event, and it is more accurate than calibrated compact text at moderate window sizes. A prompt listing only the abnormal events, the filter, is more accurate than the adapter and as fast for a single request. Its length depends on how many abnormal events a window contains, whereas the adapter's depends only on the number of events, and this decides the comparison once requests are batched. We report the result in this form because a system designer needs a rule for which lever to pull.

The contributions are as follows.

1. A communication-fidelity benchmark for the interface choice of a non-transformer sender, comparing the three efficiency levers (filter, reuse, latent) with a text baseline that includes a sender-side filter, on a task whose answer is fixed so that every error is the channel's.
2. A latent channel from such a sender into a frozen language model, extended from one event per call to fifty, in which the language model is untouched and only 0.88 million adapter parameters are trained.
3. A controlled evaluation of cost, accuracy and recoverability against a pre-specified analysis plan, including serving under batching and prefix caching, replication on a second sender architecture, and an external confirmatory test that confirmed both the adapter's advantage over calibrated text and the filter's advantage over the adapter (Section 5.4).
4. A recoverability protocol that decodes each event's label, tier, heart rate, interval and abnormal-run status from its slot of virtual tokens, separating what the channel carries from what the receiver uses.
5. A rule for choosing between a filtered text interface and a latent one, with the analysis plan, its change log and code to reproduce every table.

Section 2 places the work among the three levers and the literature on heterogeneous senders. Section 3 describes the sender, the adapter and the decoding scheme, and Section 4 the evaluation protocol. Sections 5 and 6 report and discuss the results, with limitations.

2. Related Work

We organise the related work by the three levers of Section 1, then turn to signal encoders connected to language models and to the effects that make long text prompts fail.

2.1. Saying Less

AgentPrune prunes redundant messages from the message-passing graph of a multi-agent system in one shot and reports 28.1 to 72.8% fewer tokens across existing frameworks [1]. BANDMAS splits each message into semantic packets, such as evidence and requests, and transmits only those whose predicted contribution exceeds their cost, cutting application-layer bytes by 53.2 to 77.3% on three question-answering workloads [5]. Prompt-compression methods work inside the receiver's context: they compress a prompt into a few learned tokens [6, 7], map a retrieved document to one token [8], or prune the prompt with a small language model [9].

All of these assume the content is text that the sender could have written more briefly. For a perception agent that reports structured events, the equivalent is a filter that lists only the events that matter and counts the rest. We build this baseline explicitly (Section 4). A latent channel for such a sender has to be judged against it, and it is absent from earlier comparisons of latent and text communication.

2.2. Reusing Computation and Choosing the Medium

The second lever avoids recomputing what the receiver has already read. On edge devices, persisting each agent's cache in 4-bit form lets a multi-agent system resume with large reductions in time to first token [3]. Such measures cut the cost of reading, not the number of tokens a message needs, so we report prefix caching with every arm to check that the ranking of interfaces does not depend on it.

A growing line of work treats the medium itself as a decision. Dai et al. choose between token and key–value-cache transmission according to system conditions and allocate wireless bandwidth jointly, which lowers end-to-end latency relative to either medium alone [19]. A survey of communication-efficient agentic edge intelligence formalises what to send, when and to whom as a communication policy, and names communication the primary design constraint at the edge [4]. Our selection rule (Section 6) is a policy of this kind, derived from measurements for a sender these works do not consider.

2.3. Latent Communication Between Language-Model Agents

The third lever replaces the text message with a continuous one. CIPHER transmits the expectation of the vocabulary embeddings under the sender's token distribution, so the receiver sees the sender's uncertainty as well as its sampled word [12]. LatentMAS shares last-layer hidden states and a latent working memory without any training [10]. Cache-to-cache trains a projector and a fusion module that inject one model's key–value cache into another, across model families [11]. Latent cache flow transmits a compressed cache and gives the receiver an adapter that translates and decompresses it [15]. In the bicameral model, two frozen language models are coupled through a trainable interface on their hidden states at every generation step [16]. Du et al. train a receiver-side adapter over compressed hidden states, report up to a 24-fold speed-up from compression, and call the work a feasibility study [13]. Thought communication exchanges latent thoughts instead of words [14], and reusable latent building blocks have been proposed for multi-agent pipelines [17]. Liu et al. organise these designs by carrier and injection mechanism [18].

These methods report lower cost and, in several cases, higher accuracy than text. They share a transformer sender. The transmitted object is a hidden state or a cache that a second transformer can be trained to absorb, and several methods require the two models to expose compatible internals or to be trained as a pair. The Vision Wormhole is the nearest heterogeneous case: it maps reasoning traces from different model families into a shared latent space through a universal codec and injects them into the receiver's visual pathway [20]. The heterogeneity is among vision–language models, and the sender is still a language model. A sender whose vector is a by-product of a classification loss on a physiological signal, with no language component, lies outside this literature.

A smaller body of work asks whether a latent channel is *used*. Zhang and Emu replace the message with another example's message, a compute-matched self-generated message or nothing, and show that equal end-task accuracy can hide opposite mechanisms [27]. They build on the distinction between a message that depends on the sender's state and a receiver whose behaviour depends on the message [28]. Our recoverability measurements ask a different question, what an auditor could read back from the message, and the two are complementary (Section 6).

2.4. Signal Encoders Connected to Language Models

Connecting a non-text encoder to a language model through a learned map is established practice. Visual instruction tuning projects image features into the embedding space [29], and Flamingo injects them through gated cross-attention [30]. For time series, Time-LLM keeps the language model frozen and reprograms input patches into its text-prototype space [21], and Babu et al. do the same for multichannel EEG classification [22]. SensorLM aligns wearable-sensor data and language by pretraining a CLIP- and CoCa-style model on 59.7 million hours of data [24]. In electrocardiography, ECG-Chat aligns a signal encoder with a language model for dialogue [23], ELF removes the separate encoder and lets the model read the signal directly [31], and zero-shot classification combines signal and clinical-text representations [32]. A benchmark of more than fifteen models for fetal monitoring found that fine-tuned language models performed best in most scenarios, at a higher computational cost than domain-specific architectures [33].

The frozen-model designs are closest to ours in what they hold constant, but their purpose differs. They aim to make a language model *understand* a signal and are judged on the accuracy of the resulting classifier or answer. None treats the projection as a message between two agents whose cost is compared with a text message on the same task, none compares against a sender-filtered text interface, and none reports what can be read back from the projected tokens. In our setting the perception agent is also trained for classification alone, with no language-aware objective, so the adapter is the only component that learns about the language model.

2.5. Why Long Text Prompts Degrade

Two documented effects explain why a text baseline weakens as events are added. Language models use information in the middle of a long context less well than information at its start or end

Table 1: The interfaces compared, and the efficiency lever each implements. Prefix caching is applied to every arm as a second condition and is not a separate arm.

Arm	Lever	What crosses the interface
A-full	none (reference)	Full record of every event, as text
A-compact	none (baseline)	One compact line per event, as text
A-filtered	say less	Abnormal events only, normal events as a count
Adapter	send latent	Four virtual tokens per event

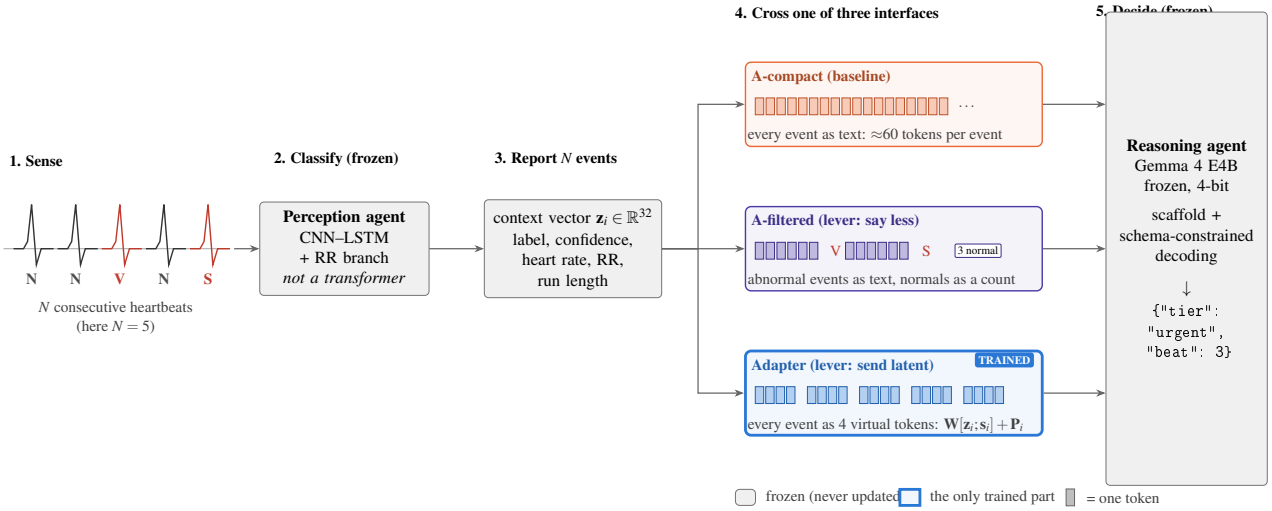


Figure 1: System overview. A perception agent that is not a transformer classifies each heartbeat, and the events of a window (abnormal beats in red) cross one of three interfaces to a frozen reasoning agent. Each small square is one token: text grows with every event (about 60 tokens each), the filter lists only abnormal events and counts the rest, and the adapter sends four virtual tokens per event. Grey boxes are frozen; only the adapter (blue) is trained. Sender, receiver and decoding are identical across arms, and the message sizes are the measured medians of Section 5.

[34], a bias that calibrating positional attention can partly correct [35]. Few-shot predictions are also biased toward certain labels, which calibrating the output scores corrects [36]. We find the label bias clearly in our text arms and only weak, inconclusive evidence of a position effect (Section 5). Following the first effect, we report text with a calibrated routine bias as the fair comparison and not the default prompt.

2.6. Gap

Saying less, reusing computation and sending latent messages have each been studied for agents that are language models. No earlier work gives a controlled, same-task comparison of these levers for a *non-transformer sender*, with a text baseline that *includes a sender-side filter*, a frozen receiver, cost measured under serving conditions, and the information a latent channel preserves measured and not assumed. The rest of the paper supplies that comparison.

3. System

Figure 1 shows the pipeline. A perception agent classifies each heartbeat and emits an event. The events of a window cross an interface to a reasoning agent, which returns a structured triage decision. Only the interface varies between arms, and each arm implements one of the efficiency levers of Section 1 (Table 1), so the comparison is one of levers on a fixed sender and a fixed receiver.

3.1. Task

Each window contains N consecutive heartbeats from one recording, with $N \in \{1, 5, 10, 20, 50\}$. The reasoning agent must return the most urgent triage tier in the window and say which beat set it.

The tier of a single beat follows a fixed escalation rule applied to the perception agent's event: *urgent* if the event requires urgent review (a run of three or more consecutive abnormal beats, or a single ventricular or fusion beat classified with confidence of at least 0.85), *routine* if the beat is classified as normal, and *priority* otherwise. The reference tier of a window is the maximum over its beats. The rule is applied to the perception agent's *predictions*, not to the annotations, because the test is whether the channel delivers the sender's output faithfully; agreement with the annotations is reported separately (Section 5). The task is a communication-fidelity benchmark, not a clinical reasoning task (Section 1).

3.2. Perception Agent

The perception agent is a convolutional–recurrent classifier trained only on the five-class AAMI EC57 beat task [37]. A 360-sample window (1 s at 360 Hz) passes through three one-dimensional convolutional blocks (32, 64 and 128 channels, with batch normalisation, ReLU and max-pooling), a bidirectional LSTM with hidden size 64, and a unidirectional context LSTM with hidden size 32. The output of the context LSTM, a 32-dimensional *context vector*, feeds the classification head and is the only representation passed downstream. There is no pretraining, no contrastive objective and no auxiliary loss.

The original design saw a single window and no timing, yet the text interface carried the RR interval (the time between successive beats), so a five-feature RR branch (previous and next RR interval, local mean RR and two ratios) was added to the context stage. Across ten training seeds it improved overall accuracy from 0.854 to 0.926 ± 0.019, ventricular sensitivity from 77% to $93\% \pm 3\%$ and the class recoverability of the context vector from 69.6% to $81.2\% \pm 5.0\%$, at the cost of one beat of latency. Fusion-beat detection stayed poor ($5\% \pm 7\%$ sensitivity). All adapters were trained on seed 0 of this encoder, which ranked third, fourth and seventh of ten on accuracy, supraventricular sensitivity and recoverability and was designated representative by a rule fixed beforehand.

A second, purely convolutional sender (residual one-dimensional convolutions without recurrence, the same RR branch and the same 32-dimensional vector) tests whether the channel depends on the first sender's architecture (Section 5).

3.3. Reasoning Agent

The reasoning agent is Gemma 4 E4B [38], loaded in 4-bit NF4 quantisation and frozen in every arm; neither adapter training nor calibration updates its weights. The model uses per-layer embeddings, a lookup keyed by token identity, which virtual tokens lack, so virtual positions receive the lookup of a padding token. This is an architectural cost of the latent channel and applies to every adapter result.

3.4. Adapter: the Latent Channel

The adapter is the third lever. Its single-event form, one linear layer from the 32-dimensional vector to four tokens with 337,920 trainable parameters, was described in the first author's MSc dissertation [39]. Here it is extended to many events and retrained, with the position embeddings, normalisation and side inputs described below. The setting imposes three requirements: the sender is unchanged and is not a transformer, the receiver is unchanged, and the message length depends only on the number of events. A linear map from the sender's own vector meets all three and keeps each event's tokens a function of that event alone. For event i with context vector $\mathbf{z}_i \in \mathbb{R}^{32}$ and three scaled side inputs $\mathbf{s}_i \in \mathbb{R}^3$ (heart rate, RR interval and abnormal-run length, each clipped and standardised), the adapter forms $\mathbf{x}_i = [\mathbf{z}_i; \mathbf{s}_i] \in \mathbb{R}^{35}$ and produces $k = 4$ virtual tokens

$$\mathbf{T}_i = \gamma \cdot \mathrm{norm}(\mathbf{W}\mathbf{x}_i + \mathbf{b} + \mathbf{P}_i) \in \mathbb{R}^{4\times 2560}, \tag{1}$$

where $\mathbf{W}$ and $\mathbf{b}$ define one linear map from 35 to 4×2560 dimensions, $\mathbf{P}_i \in \mathbb{R}^{4\times 2560}$ is a learned embedding for event position i (up to 50 positions), norm rescales each token to unit length, and γ is

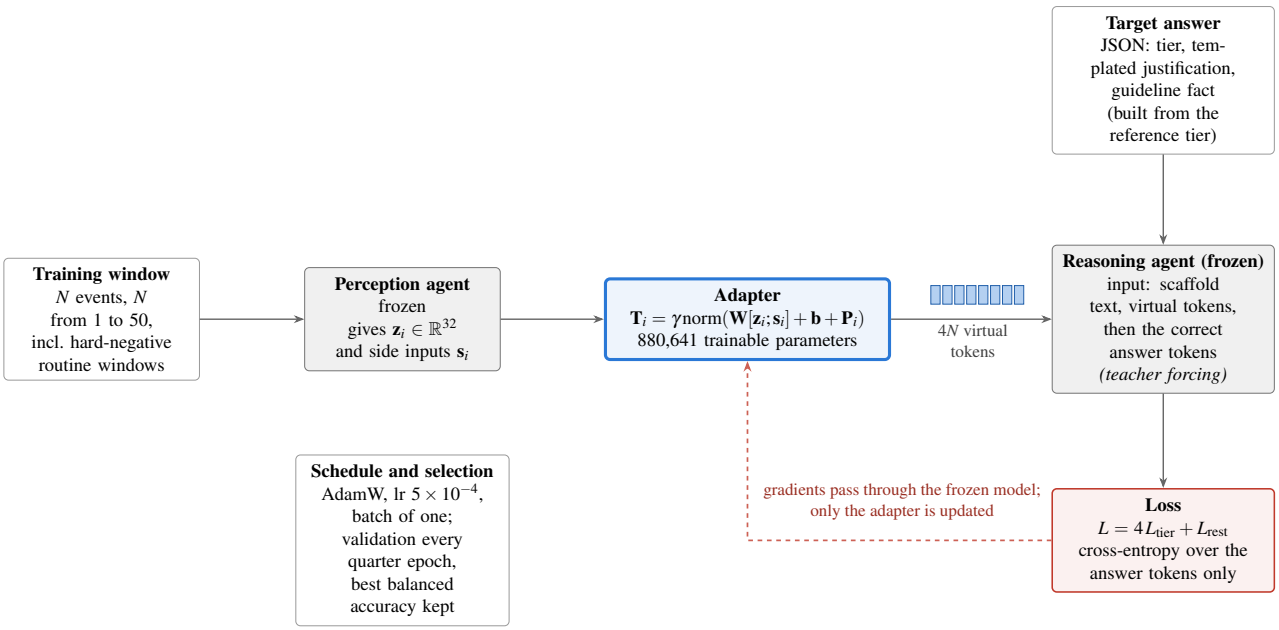


Figure 2: How the adapter is trained. The frozen perception agent supplies each event's vector and side inputs, the adapter (blue, the only trained part) turns them into virtual tokens, and the frozen reasoning agent is run on the scaffold, the virtual tokens and the correct answer (teacher forcing). The loss weights the tier tokens four times as heavily as the rest. Gradients (dashed) flow back through the frozen model but update only the adapter.

one learnable scalar initialised to the median norm of the embeddings the model computes for real tokens. Without the normalisation the norm of the virtual tokens drifted far from that of real tokens, and many of them together broke generation. The window's tokens are the $\mathbf{T}_i$ concatenated in event order, in place of the events' text, so prompt length depends only on N.

Because the map is linear and acts on one event at a time, slot i is a function of event i alone, so each event's content can be audited independently (Section 5). The map has rank at most 35, so the adapter cannot introduce information absent from its input. The adapter has 880,641 trainable parameters (358,400 in the projection weights, 10,240 in its bias, 512,000 in the position embeddings and one scale), against the roughly four billion effective parameters of the frozen model.

Each window's training target is a JSON answer in the interface schema (the urgency tier, a templated justification and a guideline fact), built deterministically from the reference tier. Training uses teacher forcing: the correct answer's tokens follow the virtual tokens, and the loss is token-level cross-entropy over the answer only, $L = 4L_{\text{tier}} + L_{\text{rest}}$, where L_{tier} averages over the tokens that spell the tier and L_{rest} over the other target tokens. Gradients pass through the frozen language model to the adapter, and only the adapter's parameters are updated. We used AdamW at a learning rate of 5×10^{-4}, warm-up over the first 5% of steps and cosine decay to 10% of the peak, gradient clipping at 1.0, a batch of one window and at most three epochs, with validation every quarter epoch, early stopping from the second epoch, and the checkpoint with the best validation balanced accuracy. The final adapter trains on 447 windows spread over N from 1 to 50 and validates on 170 windows from three held-out recordings. Figure 2 shows the flow.

The final adapter differs from an earlier version in two ways, both fixed before training. First, 72 training and 23 validation windows were added in which a normal beat is classified with confidence below 0.8 (*hard negatives*), because the earlier version raised false alarms on exactly such windows. Second, the three side inputs were added, because heart rate, interval and run length were present in the text interface but could not be recovered from the 32-dimensional vector. The two changes were made together, and an ablation separates their effects (Section 5).

3.5. Text Interfaces

All text arms use the same scaffold, instructions and decoder as the adapter arm. They differ only in how the events are written into the prompt. A-compact and A-full list every event; A-filtered is the sender-side filter, the first lever.

- **A-compact:** one line per event with class, confidence, run length and signal quality. This is the strongest text arm that lists every event and is the primary comparator.
- **A-full:** the full JSON record of each event, as in the earlier single-event study, reported only for cost.
- **A-filtered:** only the non-normal events, with their position in the window, and the number of normal events as a count. No task information is lost: the filtered window yields the reference tier on all 1,195 test windows.

A text arm that lists every event is also reported under a *calibrated* threshold (Section 4).

3.6. Schema-Constrained Decoding

Free generation produced malformed JSON in 9 to 20% of an earlier adapter's answers on the validation windows, through misnamed fields and loops. Every arm therefore uses the same constrained decoder. The structure of the answer is forced, the tier is restricted to the three valid words and chosen by the model's own scores, and free-text fields cannot close or break the structure. Every arm then produces a valid answer on every window. A check of 380 windows confirmed that the decoder changes no tier.

4. Evaluation Protocol

The protocol is designed so that a difference between arms can be attributed to the lever, not to the prompt, the model or the measurement. It fixes the sender, receiver, scaffold and decoder, measures cost on the prompt side and under serving conditions, and scores accuracy with a rule agreed before the results existed.

4.1. Analysis Plan and Change Log

The study was not recorded in a third-party study registry. Instead, the analysis plan, with its endpoints, comparators, statistics and decision rules, was committed to version control on 22 September 2026, before any run of the extended study. Every later change of design went into a numbered change log, with its date and reason, before the data it affected existed; earlier text was never edited, and analyses run after seeing results are labelled exploratory. For the external test the order is stricter: the confirmatory plan was committed before the external database was downloaded, the system was then frozen by a manifest of file hashes and a release tag, and the hash of the test-window list was committed before any model output on that database existed. Commit timestamps are self-declared, so we release the complete version history, including the change log, the freeze manifest and every result file, in an archived snapshot, so that the order of plan, data and results can be checked commit by commit. We describe this as a time-stamped analysis plan held in version control. We say where a change of design altered an outcome, and where a rule fixed in advance gave an answer we do not read literally (Section 5).

Table 2: Test windows by number of events per prompt N and reference tier (primary DS2 set, 22 records). Stratified windows are drawn per tier; natural-prevalence windows are drawn without stratification and measure false alarms at real tier frequencies. Cells shorter than their quota (for example priority at $N = 50$) are those the test records could not fill.

N	Routine	Priority	Urgent	Stratified	Natural prevalence	Total
1	60	60	57	177	—	177
5	60	59	56	175	100	275
10	60	53	56	169	100	269
20	57	45	55	157	100	257
50	40	37	40	117	100	217
All				795	400	1195

4.2. Data

Training and primary test: The MIT-BIH Arrhythmia Database [40] was used with the inter-patient split of de Chazal et al. [41]: 22 records for training (DS1) and 22 for testing (DS2), with no patient in both, after the four paced records were excluded. Of the training records, 19 were used for training and three held out entirely for validation, so that no tuning decision touched a test patient. Beats were extracted as 360-sample windows centred on the annotated R peak and standardised per window, and annotations were mapped to the five AAMI classes. The test set contains no unclassifiable beats, a property of the split.

Chronological replay: The abnormal-run count is state carried across beats of a record. Events were therefore produced by replaying each record in order from its start through the frozen perception agent, so that the run count and RR interval of every window are those a deployed system would have computed.

Test windows: A fixed sampler (seed 0) draws windows from the test records, stratified by reference tier and by the class of the window's most urgent beat, at most three per record per cell, so that urgent and priority windows are not dominated by one class. The primary set has 1,195 windows from 22 patients, of which 400 are drawn at natural prevalence to measure false alarms. Cells that cannot be filled are left short and the achieved counts are reported in Table 2.

External confirmatory data: The St Petersburg INCART 12-lead arrhythmia database [42] (75 half-hour recordings from 32 patients) was fixed as the confirmatory test before it was downloaded or inspected. Lead II is resampled from 257 to 360 Hz and converted with the same procedure, and no INCART data are used for training, tuning or prompt design. The adapter, the text prompts and the calibration biases are frozen and hashed in a manifest before the run. The results are reported in Section 5.4.

4.3. Comparators and Calibration

The comparators are A-compact, A-full (cost only) and A-filtered (Table 1). Reusing computation, the second lever, is not an arm: prefix caching is switched on and off for every arm in the serving measurements, to check the ranking of the other levers under it. A text model with every event listed is biased toward answering "routine" on long prompts, an effect known from few-shot prompting [36]. To avoid a strawman we report each text arm under default decoding and with a bias added to the routine score at the tier step, chosen on the held-out validation windows only and then frozen. The calibrated arm is the fair comparison and is used for every inferential statement.

4.4. Endpoints and Statistics

Accuracy: The primary endpoint is balanced accuracy, the mean recall over the tiers present, so that the rare urgent tier carries the same weight as the common routine tier. Arms are compared on the same windows (paired). Because windows from one patient are correlated, confidence intervals use a

patient-cluster bootstrap with 20,000 resamples and percentile 95% limits. The primary question for the adapter against calibrated text is non-inferiority with a margin of -0.05, tested first, followed by superiority where non-inferiority holds, in a fixed order at $\alpha = 0.05$. Adapter results are the mean of three training seeds, and seed ranges are reported.

False alarms: The share of routine windows at natural prevalence that the arm escalates to priority or urgent.

Cost: Token counts of the prompt; time to first token (prompt processing) and time to the tier token (the decision), at batch size one with arm order rotated; context memory; and GPU energy. We report time to the decision, not time to the full answer, because answer length differs across arms while decode speed is identical (about 160 ms per token); the full answer would credit the interface with differences in the length of the justification. Cost medians use a paired bootstrap of the relative difference on 21 windows per N.

Serving: Throughput and energy per decision were measured with batches of one, four and eight, repeated three times, and with prefix caching on and off. The framework was capped at 90% of the card's memory so that a batch that does not fit is recorded as such instead of being silently paged into system memory, which would measure paging and not the model. Energy is the GPU board power reported by the NVIDIA Management Library, sampled at 100 Hz during each timed call (prompt processing and generation up to the tier token) and integrated over its wall-clock time; energy per decision is the batch's energy divided by the batch size. The measurement covers the whole board, including its idle and static draw, and excludes the processor, host memory and the rest of the system; every call spanned at least 87 power samples. Each configuration was measured on every window chunk in each of the three repeats (63 measurements per cell at batch one, 15 at batch four and 6 at batch eight), and we report the median with the interquartile range. Serving was measured for one adapter (the first training seed), so the spread reflects measurement and content variation, not training variation.

Recoverability: For each event, a decoder is trained on the training records to predict label, tier, heart rate, RR interval and abnormal-run status from the virtual tokens of a given slot, and tested on the test records. The same decoder family is trained on the 32-dimensional input and on the 35-dimensional input including side inputs, so that the loss attributable to the channel is the gap between the two. Decoders are fitted per slot (slots 0, 9, 19 and 49). The decoder fixed in advance is a small multilayer perceptron; a linear decoder is fitted alongside it for comparison. Run status is scored as balanced accuracy for a run of three or more beats, because R^2 is ill-posed for the heavy-tailed run length on this split (a change recorded before any data from it were analysed).

4.5. Implementation and Reproducibility

All experiments ran on one workstation with an NVIDIA RTX 5070 (12 GB), Python 3.13 and HuggingFace `transformers` with `bitsandbytes`. Production serving engines do not accept continuous input embeddings, so absolute latencies are higher than an optimised engine would give; every arm shares this, so only relative comparisons are meaningful. Every run records its git commit, checkpoint hashes, seed and library versions. Adapter training checkpoints resume bit-identically, and one run was paused and resumed without effect on the result. The tables and figures are generated from the result files by scripts in the supplementary release.

5. Results

All numbers in this section come from the evaluation on the MIT-BIH test set (DS2) fixed in the analysis plan (1,195 windows, 22 patients) unless stated otherwise, and the tables and figures are generated from the result files by the scripts in the supplementary release. Adapter values are means of three training seeds. Intervals are 95% patient-cluster bootstrap limits.

Table 3: Cost of the adapter relative to compact text (batch one, median over 21 windows per N). TTFT: time to first token; Decision: time to the tier token; Context MB: change in context memory.

N	Tokens (text / adapter)	TTFT ms (text / adapter)	TTFT	Decision	Context MB
1	576 / 516	181 / 184	0%	+1%	—
5	811 / 532	197 / 182	−8%	−1%	−36
10	1105 / 553	241 / 183	−24%	−4%	−79
20	1694 / 593	360 / 181	−50%	−14%	−177
50	3457 / 713	845 / 183	−78%	−36%	−460

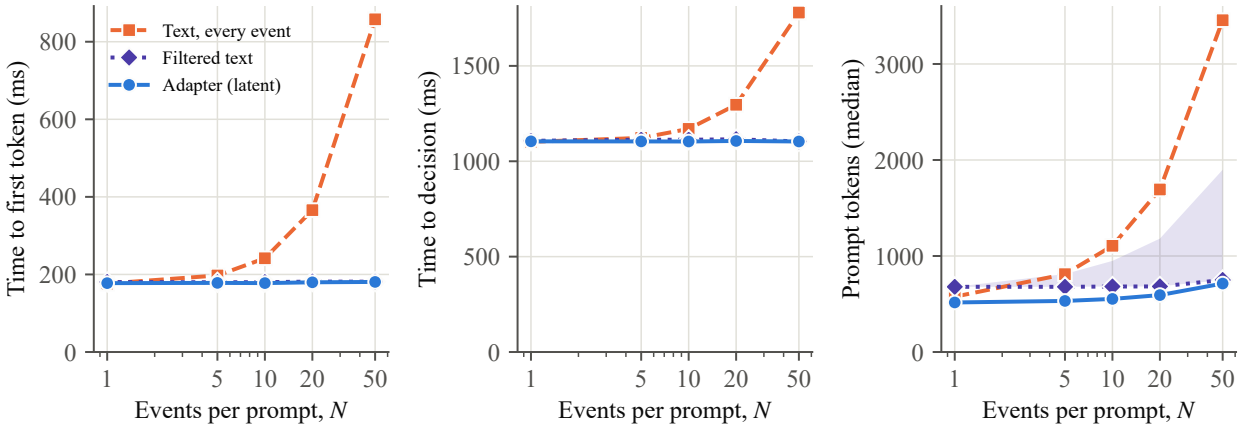


Figure 3: Cost of the three interfaces against the number of events per prompt (batch one, median of 21 windows per N). The shaded band in the right panel is the 90th percentile of the filtered prompt: its length depends on content, whereas the adapter's depends only on N. Time to decision is almost identical for filtered text and the adapter.

The subsections answer the three questions of Section 1 in turn: what the latent lever costs relative to the others, what it gives up or gains in the decision, and what can still be read back from the message. Each comparison is against the strongest text arm available for the question: calibrated compact text for accuracy against listing every event, and filtered text for any claim that the latent channel is worth building. Section 6 combines the three answers into a selection rule.

5.1. Cost of the Interface

At a single event the adapter has nothing to save. Its four tokens replace about 57 tokens of compact text, but the fixed scaffold dominates the prompt (end-to-end lengths of 486 and 458 tokens for the two arms in the single-event study), and time to first token is unchanged. The advantage appears only when a prompt carries many events (Table 3). Text grows by about 60 tokens per event whereas the adapter adds four, so the adapter's prompt-processing time stays near 183 ms from 1 to 50 events while the time of compact text rises to 845 ms. Time to the tier token falls less because the decision includes generating the first answer tokens, which costs the same in both arms.

Serving: Under batching, text that lists every event could not be served at a batch of eight at any $N \geq 10$, nor at a batch of four at 50 events, on the 12 GB card, whereas the adapter fitted every configuration with a peak of about 10 GB. Against A-filtered, the adapter was within about 5% in time and throughput for single requests, with 3 to 12% less energy per decision and overlapping interquartile ranges. Once requests were batched, its median throughput was 10 to 30% higher and its median energy per decision 12 to 28% lower (Table 4), but at a batch of four the interquartile ranges of the two arms overlap for throughput at 10 and 50 events and for energy at 50 events, because the cost of filtered text varies with content. Filtered text fitted only some batches: 3 of 6 at a batch of eight for every N, and 12 of 15 at a batch of four for 50 events. Its figures for those cells describe only the

Table 4: Decisions per second and GPU board energy per decision when requests are batched: median (interquartile range) over all measurements of the cell (final adapter, first training seed). "Does not fit": the batch exceeds the 12 GB memory cap. *Only the stated number of batches fitted; the values describe that subset (the shortest prompts) and not the arm at that batch size.

N	Batch	Adapter	Text, all events	Filtered text
10	1	0.98 (0.94–1.02)/s; 83 (77–87) J	0.92 (0.82–0.97)/s; 111 (103–120) J	0.97 (0.83–1.02)/s; 95 (86–105) J
10	4	3.19 (2.87–3.32)/s; 38 (38–42) J	2.23 (2.19–2.36)/s; 65 (64–67) J	2.80 (2.11–2.94)/s; 49 (45–70) J
10	8	4.72 (4.50–4.86)/s; 33 (31–33) J	does not fit	3.63 (3.48–3.77)/s; 43 (42–44) J* 3/6
20	1	0.98 (0.91–1.05)/s; 81 (77–88) J	0.85 (0.78–0.88)/s; 134 (129–143) J	0.97 (0.85–1.01)/s; 92 (83–108) J
20	4	3.17 (3.10–3.30)/s; 39 (38–40) J	1.70 (1.69–1.74)/s; 92 (90–92) J	2.49 (1.87–2.72)/s; 54 (47–77) J
20	8	4.55 (4.13–4.75)/s; 33 (32–36) J	does not fit	4.00 (3.95–4.11)/s; 38 (38–39) J* 3/6
50	1	1.00 (0.87–1.06)/s; 89 (83–96) J	0.60 (0.56–0.61)/s; 232 (227–244) J	0.96 (0.83–1.00)/s; 92 (85–110) J
50	4	2.79 (2.49–2.92)/s; 48 (46–52) J	does not fit	2.25 (1.78–2.89)/s; 63 (47–83) J* 12/15
50	8	4.11 (3.97–4.27)/s; 38 (37–39) J	does not fit	3.72 (3.66–3.79)/s; 43 (42–43) J* 3/6

Table 5: Balanced accuracy on class-balanced DS2 windows, and adapter minus calibrated text with 95% patient-cluster intervals. Rule: the reference rule applied directly to the sender's outputs, 1 by construction and computed in microseconds; it is the ceiling of every channel and shows that the task itself does not need a language model (Section 1). The adapter is superior at $N = 10$ and 20 (interval above zero) and non-inferior at $N = 5$ and 50 (lower limit above the margin of -0.05).

N	Rule	Text	Text cal.	Adapter	Adapter − cal. text [95% CI]
1	1.00	0.97	0.97	0.80	—
5	1.00	0.73	0.79	0.81	+0.03 [−0.04, +0.10]
10	1.00	0.56	0.73	0.83	+0.10 [+0.03, +0.17]
20	1.00	0.48	0.73	0.83	+0.10 [+0.01, +0.18]
50	1.00	0.54	0.76	0.84	+0.08 [−0.02, +0.17]

batches that fit, which are its shortest prompts, so they favour filtered text and are not a measurement of filtered text at that batch size. These serving results come from one GPU, one serving framework and one adapter, and we read them as a promising result for this setup rather than a general advantage. Prefix caching shortened every arm's prefill and did not change the ranking.

5.2. Accuracy

Against text that lists every event: Default compact text falls from 0.97 balanced accuracy at one event to 0.48 to 0.56 at 10 to 50 events. Calibration recovers part of this (0.73 to 0.76) and is the fair comparison. The final adapter holds 0.81 to 0.84 at 5 to 50 events (Table 5). Against calibrated text, the adapter was superior at 10 and 20 events (both +0.10, intervals excluding zero) and non-inferior at 5 and 50 events. At a single event, text was better (0.97 against 0.80), as expected from the cost analysis.

Why text with every event fails: Two mechanisms account for most of the loss (Figure 6). Text misses urgent windows caused by a single high-confidence ventricular or fusion beat (a numeric threshold of 0.85 buried in a long list) far more often than windows caused by a run of abnormal beats. This is the clearest effect: text recalls 0.17 of such windows against 0.82 for the adapter. The position of the deciding beat points the way reported for long contexts [34, 35]: text accuracy is lowest when that beat is in the middle third of the window (0.19, against 0.30 in the first and 0.38 in the last third). The evidence is weak. The middle group has 73 windows and its interval overlaps those of the other thirds (right panel of Figure 6), and the adapter shows a similar small dip (0.74 to 0.78 across positions). We report a position effect as suggested for text, not established, and not distinguishable from the adapter's.

False alarms: The earlier adapter escalated about 11% of routine windows, concentrated on windows with a normal beat of low classification confidence. The final adapter reduced the rate at every seed,

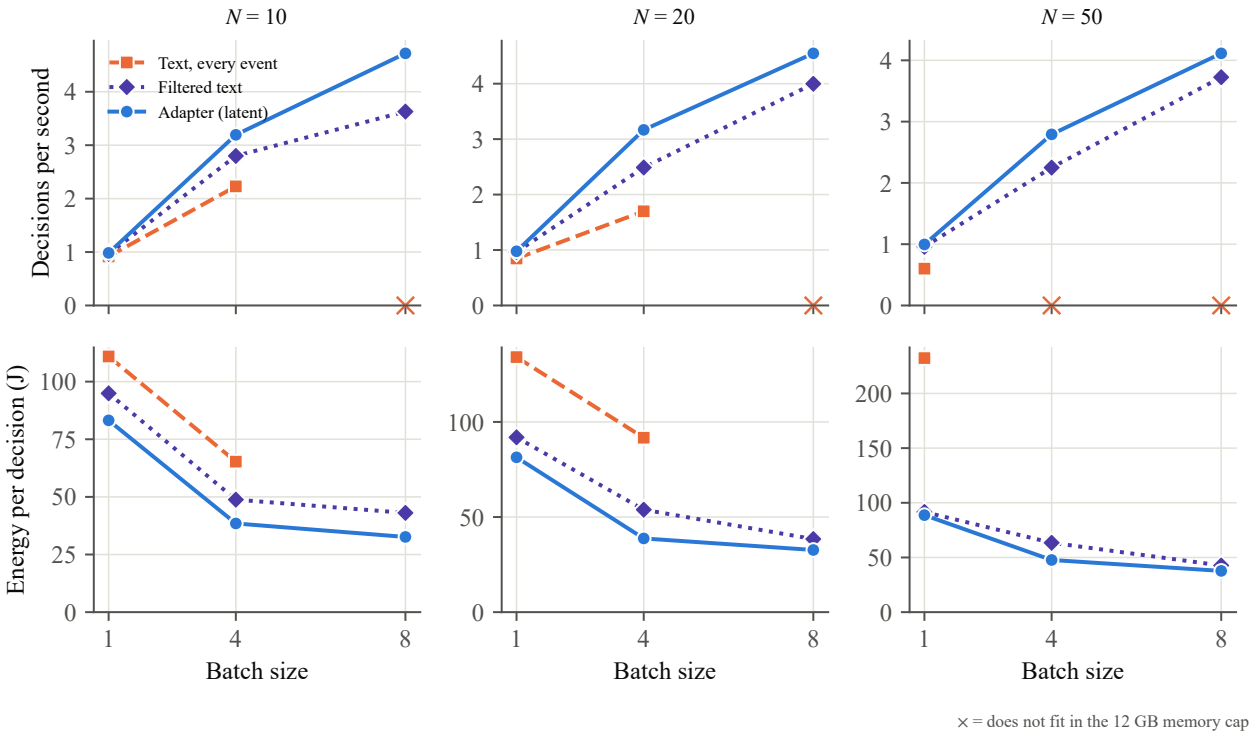


Figure 4: Throughput (top) and GPU energy per decision (bottom) when requests are batched, for 10, 20 and 50 events per prompt (final adapter, first training seed; medians). A cross marks a configuration that did not fit in the memory cap of the 12 GB card. Filtered text at batch eight is plotted from the batches that fit, which are its shortest prompts.

Table 6: Ablation of the final adapter (three seeds each): balanced accuracy, false alarms (FA) on routine windows, and the two contrasts with patient-cluster 95% intervals. Hard negatives: hard-negative routine windows in training; side inputs: heart rate, interval and run length. The side-input contrast is clean (same windows and schedule); the hard-negative contrast is approximate, because the earlier adapter also used fewer training windows and a proportionally shorter schedule.

Adapter	N=5	N=10	N=20	N=50	FA
Earlier adapter (neither)	0.794	0.772	0.795	0.770	10.6%
Hard negatives only	0.788	0.786	0.802	0.781	8.6%
Final adapter (both)	0.814	0.830	0.830	0.836	3.6%
Side inputs: final − hard negatives only	+0.03 [−0.02, +0.06]	+0.04 [−0.01, +0.11]	+0.03 [−0.01, +0.07]	+0.05 [+0.01, +0.11]	
Hard negatives: hard negatives only − earlier	−0.01 [−0.04, +0.03]	+0.01 [−0.02, +0.05]	+0.01 [−0.04, +0.05]	+0.01 [−0.03, +0.05]	

by 44 to 80% (about two thirds on average), to 3.6% overall (0% to 3.8% at 50 events). Text raised none, because its error is the opposite one: it under-escalates urgent windows to priority. The adapter never called an urgent window routine in the 50-event confusion matrix.

What the final adapter's gain comes from: The final adapter differs from the earlier one in two ways: hard-negative routine windows in training, and three side inputs per event (heart rate, interval and run length). We retrained it without the side inputs, with three seeds and everything else identical (Table 6). The side inputs account for most of the improvement. Accuracy rose at every N (by 0.03 to 0.05, with the interval excluding zero at 50 events) and false alarms fell from 8.6% to 3.6% in every seed. Hard negatives alone moved accuracy by about 0.01 and false alarms from 10.6% to 8.6%. The false-alarm reduction therefore comes mainly from giving the receiver the context to tell a low-confidence normal beat from a real event, not from the extra training examples.

Against text that lists only abnormal events: A-filtered was more accurate than the adapter at every N (Table 7), raised no false alarms, and for a single request matched it in time to first token and to decision within 1.5%. Its cost depends on content: each listed abnormal beat adds about 68 tokens, so at 50 events its prompt is longer than the adapter's in half of the windows. The decision rule fixed in the analysis plan anticipated a filtered prompt that was cheaper but less accurate; applied literally it

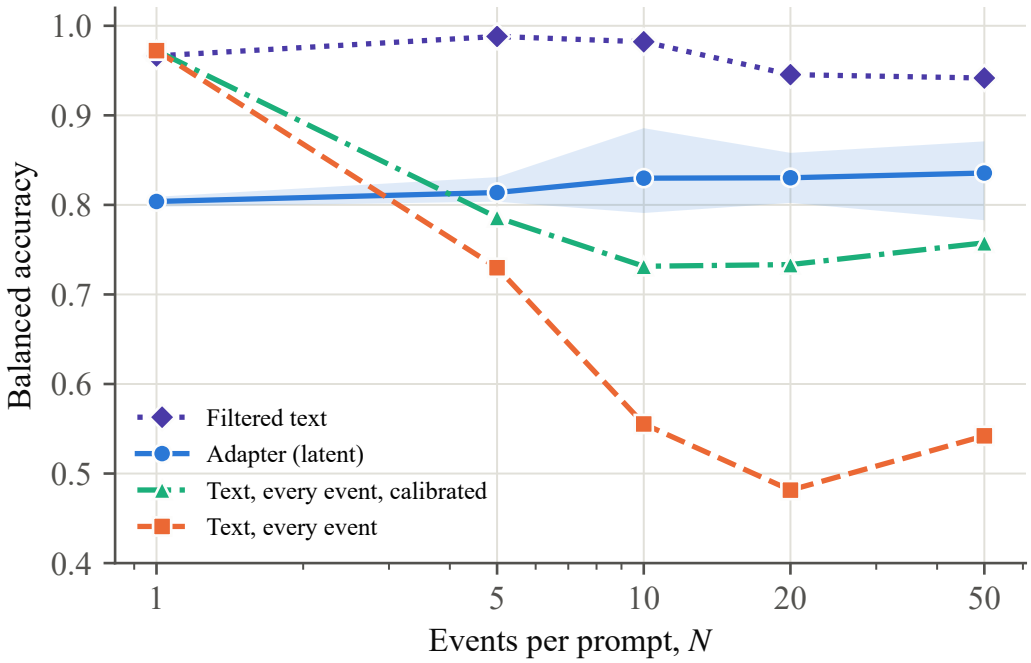


Figure 5: Balanced accuracy against the number of events per prompt on the class-balanced test windows. The shaded band is the range of the three adapter seeds. Filtered text is the most accurate arm; text with every event listed degrades as events are added and calibration recovers only part of the loss.

Table 7: Adapter against filtered text (default decoding) with patient-cluster intervals, and median prompt tokens (adapter / filtered).

N	Adapter	Filtered	Difference [95% CI]	Tokens
1	0.804	0.966	—	516 / 679
5	0.814	0.988	−0.17 [−0.22, −0.13]	532 / 679
10	0.830	0.982	−0.15 [−0.19, −0.11]	553 / 682
20	0.830	0.945	−0.12 [−0.16, −0.07]	593 / 683
50	0.836	0.942	−0.11 [−0.17, −0.05]	713 / 750

returned "advantage holds" because the filtered prompt turned out to be longer. We do not read it that way. Against filtered text, on this task, the latent interface has no accuracy advantage and no latency advantage for a single request.

Against the annotations: Scored against the database annotations instead of the sender's own rule, with a window counted as abnormal if any of its beats is annotated as not normal, the reference rule has sensitivity of 0.94 to 0.96 and specificity of only 0.46 to 0.64 across window sizes. The clinical ceiling is therefore set by perception and not by the interface: the final adapter tracked the reference (sensitivity 0.90 to 0.96 at 5 to 50 events), while default compact text missed more abnormal windows (0.77 to 0.79 at 10 to 50 events).

Second sender: The replication with a convolutional sender (validation accuracy 0.955 against a gate of 0.85) reproduced the pattern (Table 8). Text with every event listed fell as events were added while the adapter stayed at 0.77 to 0.82; the adapter was superior to calibrated text at 10 events and non-inferior at 20 and 50, but not at 5; false alarms were 1.3 to 2.1%. A-filtered was again the most accurate arm except at 50 events, where it fell to 0.80 and the difference was inconclusive (−0.01 [−0.09, +0.08]). An exploratory look at those windows (40 per sender, so indicative only) suggests that this sender's urgent windows carry sparser, more borderline evidence, often one urgent

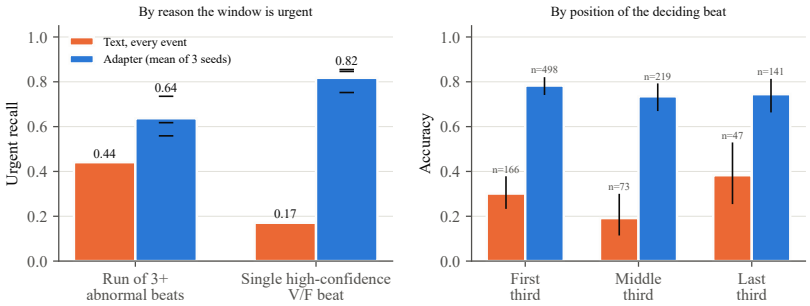


Figure 6: Urgent recall by the reason a window is urgent. Text with every event listed misses windows made urgent by a single high-confidence ventricular or fusion beat far more often than windows made urgent by a run; the adapter does not show this gap. Bars for the adapter are the mean of three seeds, with each seed's value marked. Right: accuracy on non-routine windows with $N \geq 10$ by the position of the window's most urgent beat; whiskers are 95% Wilson intervals, and the adapter counts are windows times three seeds. This panel is exploratory (not in the analysis plan).

Table 8: Second sender (ResNet1D-RR): balanced accuracy on its own DS2 windows and adapter minus calibrated text (non-inferiority margin -0.05).

N	Adapter	Text cal.	Filtered	Adapter − cal. text [95% CI]
5	0.770	0.787	0.959	-0.02 [-0.11, $+0.08$]
10	0.823	0.732	0.929	$+0.09$ [$+0.02$, $+0.16$]
20	0.780	0.733	0.907	$+0.05$ [-0.03, $+0.13$]
50	0.794	0.733	0.808	$+0.06$ [-0.02, $+0.14$]

beat among about 13 listed abnormal beats with a deciding confidence just above 0.85. Filtered text depends on how the sender's evidence is distributed, whereas the adapter did not change much.

External confirmatory test: The frozen system was evaluated once on a second database; the result is reported in Section 5.4.

5.3. What Can Be Recovered

Single events: Decoders trained on the training records and applied to the test records recovered the predicted beat label (accuracy 99.5%) and tier (95%; balanced accuracy 0.88) from the virtual tokens of the single-event adapters as well as from the 32-dimensional input, so the adapter loses nothing the vector holds.

Every position: For slots 0, 9, 19 and 49 of a 50-event window, label, tier and urgency decoded at the level of the input vector. Decoders are slot-specific: a decoder trained on slot 0 applied at slot 49 fell to 0.64 to 0.91, so an auditor needs a position-aware decoder.

What the vector never carried: The 32-dimensional context vector does not contain heart rate, interval or run length (R^2 about 0.3), so the earlier adapter could not carry them either. With the side inputs of the final adapter they are recoverable from the tokens (Table 9). The decoder specified in advance was a small multilayer perceptron, and with it heart rate (R^2 0.59 to 0.78) and interval (0.65 to 0.77) fall well short of the input (0.98 and 0.95), while the presence of a run of three abnormal beats is recovered at 0.93 to 0.98 balanced accuracy (0.53 to 0.61 for the earlier adapter, near chance). A linear decoder, fitted in the same analysis, recovers much more of the numeric fields from the same tokens (heart rate 0.95 to 0.97, interval 0.92 to 0.96). The multilayer decoder therefore understates what the tokens hold, and the information is held in a nearly linear form. We report both: the figure from the decoder specified in advance is the conservative one, and the linear figures are the better estimate of what an auditor could recover. On the second sender, heart rate and interval are recovered from every token slot at R^2 0.89 to 0.94 with the decoder specified in advance. Signal quality is not carried by either sender.

Table 9: Recoverability of each event's fields from its virtual tokens (final adapter, main sender), decoders trained on the training records and applied to the test records. Token columns give the range over slots 0 and 49 and three seeds. The multilayer perceptron was the decoder specified in advance; the linear decoder is reported for comparison.

Field	Input, MLP	Tokens, MLP	Input, linear	Tokens, linear
Beat label (bal. acc.)	0.99	0.95–0.98	0.99	0.99–1.00
Tier (bal. acc.)	0.95	0.92–0.95	0.91	0.90–0.94
Urgency flag (bal. acc.)	0.98	0.97–0.98	0.94	0.94–0.95
Heart rate (R^2)	0.98	0.59–0.78	1.00	0.95–0.97
RR interval (R^2)	0.95	0.65–0.77	0.98	0.92–0.96
Run ≥ 3 (bal. acc.)	0.98	0.93–0.98	0.95	1.00–1.00

Table 10: Virtual tokens per event: three seeds for k = 1, 2 and 4 (means), one run for k = 8. Tokens: prompt length at 50 events. $k-4$: difference from k = 4 at the least favourable N (50 in both cases), with its patient-cluster 95% interval; NI: number of N (of 5, 10, 20, 50) at which k is non-inferior (margin -0.05). FA: false alarms on routine windows (seed ranges in the text). Label and HR R^2: recoverability from slot 0 with the decoder specified in advance (multilayer perceptron); the negative R^2 at k = 8 is likely a decoder limit (32 principal components of a 20,480-dimensional slot).

k	Tokens	Bal. acc., N = 5/10/20/50	$k-4$ [95% CI]	NI	FA	Label	HR R^2
1	563	0.70 / 0.72 / 0.75 / 0.73	-0.11 [-0.16, -0.06]	0/4	6.3%	0.97	0.87
2	613	0.80 / 0.82 / 0.83 / 0.80	-0.04 [-0.13, $+0.03$]	1/4	8.6%	0.95	0.81
4	713	0.81 / 0.83 / 0.83 / 0.84	ref.	–	3.6%	0.96	0.71
8	913	0.77 / 0.80 / 0.78 / 0.80	1 run	–	6.1%	0.95	-0.62

Carried and used: The ablation shows both directions. Without the side inputs, heart rate, interval and run length are not recoverable from the tokens at all (R^2 at most 0.09; run ≥ 3 at 0.52 to 0.59, near chance); with them they are, and the receiver uses them, since they account for most of the accuracy gain and the false-alarm reduction (Table 6). Use is partial, however: urgent recall on windows made urgent by a run of three or more abnormal beats rose only from 0.61 to 0.64 when run length entered the tokens, against 0.44 for text.

Pre-generation check: For the single-event adapters, scores computed from the context vector alone flag inputs before generation: a tier decoder predicts the adapter's wrong answers (AUROC 0.89 to 0.97), and a Mahalanobis distance flags noise and powerline interference (0.99). This gives an inspection that does not need to read the tokens.

Compression: We retrained the adapter with one, two and eight virtual tokens per event in place of four (Table 10, Figure 7). A first sweep with one seed per setting suggested that a single token might suffice, so one and two tokens were repeated with three seeds each, under a rule fixed before training: a setting is free of accuracy cost only if it is non-inferior to four tokens (margin -0.05) at every N. Neither passed. With one token, one run in three failed to learn the task (0.49 to 0.55 at 5 to 50 events) while the other two matched four tokens, so the three-seed mean was 0.09 to 0.11 lower. With two tokens accuracy was within about 0.01 of four tokens at 5 to 20 events and 0.04 lower at 50, non-inferior only at 20 events, and false alarms were higher in every run (8.6% on average against 3.6%). Eight tokens, a single run, was no better than four. Each event's label, heart rate and run status remained recoverable from a single token, including from the one-token run that failed the task (label 0.96 to 0.99, heart-rate R^2 about 0.9 with the decoder specified in advance). Compression below four tokens per event therefore preserved what the tokens carry but made the receiver's use of it unreliable, and four tokens was the smallest setting at which every run was reliable. The saving would be modest in any case, because the scaffold dominates: at 50 events, 563 tokens at one token per event against 713 at four.

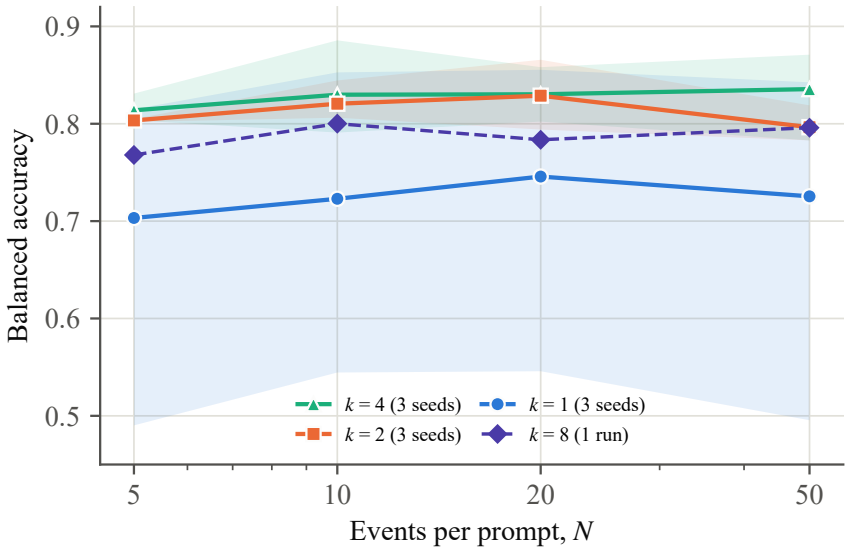


Figure 7: Balanced accuracy against the number of events for $k = 1$, 2 and 4 virtual tokens per event (lines: mean of three seeds; bands: seed range) and $k = 8$ (one run). The wide band at $k = 1$ is one failed run.

Table 11: INCART test windows by number of events per prompt N and reference tier. All stratified cells reached their quota.

N	Routine	Priority	Urgent	Stratified	Natural prevalence	Total
1	60	60	60	180	—	180
5	60	60	60	180	100	280
10	60	60	60	180	100	280
20	60	60	60	180	100	280
50	40	40	40	120	100	220
All				840	400	1240

5.4. External Confirmatory Test on INCART

The DS2 test set informed several design iterations, so its results cannot confirm the final system. The confirmatory test is therefore a single run of the frozen system on a database that played no part in any decision. The St Petersburg INCART database has 75 half-hour recordings from 32 patients; after conversion to the same beat format (lead II, resampled from 257 to 360 Hz) it yields 175,777 accepted beats: 153,594 normal, 20,006 ventricular, 1,958 supraventricular and 219 fusion beats. A further 91 beats whose window fell outside the recording and 51 with an unmapped annotation symbol were excluded. The test set has 1,240 windows with every stratified cell filled (Table 11), and its hash was committed before any model output existed.

The system under test was fixed in a manifest of file hashes before the run: the three final adapters (not retrained), the perception agent, the prompt and decoding code, and the calibration biases chosen on the DS1 validation windows and not re-tuned. Text arms were generated by stopping at the tier token, as decided after that choice was shown on DS2 to change no tier. The four hypotheses were tested in a fixed sequence at $\alpha = 0.05$, stopping at the first failure, with patient-cluster bootstrap intervals (20,000 resamples): **H1**, the adapter (three-seed mean) is non-inferior to calibrated compact text at $N = 10$, 20 and 50 (margin -0.05, all three required); **H2**, it is superior at $N = 10$ and 20; **H3**, it is non-inferior to calibrated filtered text at $N = 10$, 20 and 50; **H4**, its median time to first token is lower than that of compact text at $N = 10$, 20 and 50. Hypotheses after the first failure are reported descriptively as not tested in sequence.

Both hypotheses on accuracy against calibrated compact text were confirmed (Table 12). The adapter's three-seed mean was non-inferior at 10, 20 and 50 events (H1) and superior at 10 and 20

Table 12: Hypotheses on INCART, fixed in advance. Estimates are adapter minus calibrated text (H1, H2), adapter minus calibrated filtered text (H3), or the median relative difference in time to first token (H4), with 95% intervals. H4 was not tested in sequence because H3 failed. †The time-to-first-token value at 50 events is the technical re-measurement under the memory cap (see text); the original value (−0.97) was distorted by memory paging.

Hypothesis	*N*	Estimate	95% CI	Result	Status
H1: non-inferior to calibrated text	10	+0.08	[+0.03, +0.15]	pass	confirmed
	20	+0.09	[+0.04, +0.15]	pass	
	50	+0.05	[−0.04, +0.13]	pass	
H2: superior to calibrated text	10	+0.08	[+0.03, +0.15]	pass	confirmed
	20	+0.09	[+0.04, +0.15]	pass	
H3: non-inferior to calibrated filtered text	10	−0.11	[−0.17, −0.05]	fail	not confirmed
	20	−0.11	[−0.16, −0.06]	fail	
	50	−0.07	[−0.15, −0.00]	fail	
H4: faster first token than text	10	−0.27	[−0.28, −0.26]	pass	not tested in sequence
	20	−0.51	[−0.52, −0.49]	pass	
	50†	−0.77	[−0.78, −0.77]	pass	

Table 13: Balanced accuracy on INCART by number of events per prompt (adapter: mean of three seeds). Rule: the reference rule on the sender's outputs, 1 by construction.

N	Rule	Adapter	Text	Text cal.	Filtered	Filtered cal.
1	1.000	0.746	0.983	0.978	0.972	0.972
5	1.000	0.789	0.722	0.806	0.939	0.933
10	1.000	0.815	0.583	0.733	0.939	0.922
20	1.000	0.822	0.517	0.728	0.933	0.928
50	1.000	0.822	0.533	0.775	0.892	0.892

events (H2): the differences were +0.08 [+0.03, +0.15] at 10 events and +0.09 [+0.04, +0.15] at 20 events, with patient-cluster intervals, and the margin at 50 events was cleared narrowly (+0.05 [−0.04, +0.13] against a margin of −0.05). These estimates are close to those on the MIT-BIH test set (+0.10 at both 10 and 20 events), on a different database, a different lead resampled from 257 to 360 Hz, and different patients. The result for filtered text was the opposite, and H3 was not confirmed. Filtered text was the most accurate arm at every number of events above one (0.89 to 0.94 against 0.82 for the adapter at 10 to 50 events, Table 13), and the adapter was inferior to it at every tested size: −0.11 at 10 and 20 events and −0.07 at 50 events, with every interval excluding zero (at 50 events only just, [−0.15, −0.004]). Following the rule fixed in advance, H4 is therefore reported as not tested in sequence. It is described below.

Accuracy by window size: The pattern matches MIT-BIH. At a single event text is more accurate than the adapter (0.98 against 0.75). Text that lists every event degrades as events are added, from 0.72 at 5 events to 0.53 at 50 events under default decoding, and calibration recovers part of the loss (0.73 to 0.81). The adapter stays between 0.79 and 0.82 at 5 to 50 events, with a seed range of 0.79 to 0.84 at 10 events (Table 13).

False alarms and parsing: Every arm produced a valid answer on every window (parse rate 100%). Text and filtered text raised no false alarms. The adapter escalated 0.2%, 3.3% and 1.1% of routine windows for the three seeds, the same range as or lower than the 2.7 to 4.8% of the three seeds on MIT-BIH. In 11 to 16% of windows the adapter's free-text fields reached the fixed length cap; the field-cap rate is undefined for the text arms, which were stopped at the tier token.

Time to first token (descriptive): The adapter's prompt-processing time stayed near 180 to 200 ms at every number of events, as on MIT-BIH, while that of compact text rose with the prompt. The median relative difference was −27% at 10 events and −51% at 20 events, within a few points of the MIT-BIH values (−24% and −50%), and each interval excluded zero. At 50 events the confirmatory

Table 14: Routine windows escalated to priority or urgent on INCART.

Arm	All routine windows	Natural prevalence
Adapter, run 1	0.2%	0.0%
Adapter, run 2	3.3%	2.8%
Adapter, run 3	1.1%	1.1%
Text, calibrated	0.0%	0.0%
Filtered, calibrated	0.0%	0.0%

run's timing of compact text was not usable: it took a median 6.9 s, with a range of 3.3 to 15.3 s across the 21 windows, against 0.85 s for a prompt of the same length on MIT-BIH, which is the signature of memory paging on the 12 GB card, a condition controlled in the serving measurements with a memory cap (Section 4) but not in this timing routine. We therefore re-measured that step alone, documented before the measurement as a technical change to the protocol: the same frozen routine, on the same 21 windows, with the memory cap applied and cached memory released before each timed call, outside the timer. Compact text then took a median 859 ms (maximum 881 ms), the adapter 194 ms and filtered text 195 ms, so the adapter was -77% [-78%, -77%] faster than compact text (MIT-BIH: -78%) and not different from filtered text (-2% [-15%, $+3\%$]). The original measurement is kept on record. H4 remains not tested in sequence, and these figures are descriptive.

What the confirmatory test establishes: On a second database that played no part in any decision, the accuracy advantage of the adapter over calibrated compact text replicated at 10 and 20 events, and the advantage of a sender-side filter over the adapter replicated at every number of events. The test does not show that the adapter beats the best text interface, and it does not test the unknown-question case, for which no experiment here has a fixed answer. Perception accuracy on INCART is not an endpoint and is not reported here.

6. Discussion

6.1. Choosing a Communication Interface

The three questions give three answers, and they do not point to the same arm. Table 15 sets the arms side by side on the criteria a system designer would use. Reading across it, no interface dominates: the filter wins on accuracy and readability, the latent channel on predictability of cost under load, and listing every event on nothing except simplicity at small N.

From the table we draw a selection rule for the task studied, with one part stated as a hypothesis.

- *The receiver's question is fixed and known when the sender writes its message:* filter on the sender side and send text. The filter was the most accurate arm at almost every N on both senders and on the external database, raised no false alarms, matched the adapter for a single request, and can be read by a person without any decoder.

- *Many events must be communicated at a predictable cost, or requests are batched on constrained hardware:* use the latent channel. Its length depends only on N, so memory and batch size can be planned in advance, and in the batched measurements it delivered 10 to 30% more decisions per second at 12 to 28% less energy per decision than filtered text.

- *The question is not known when the sender encodes* (hypothesis, not tested here): a filter cannot discard what it cannot rank, whereas the latent channel carries information about every event. Our task has one question, so the experiment cannot show that the extra information would be used by a different one. A study that varies the question over fixed virtual tokens is the direct test, and we leave it to future work.

Table 15: Interfaces compared on the criteria that decide the choice, from the measurements in Section 5. "Content-dependent" means that the cost varies with how many abnormal events a window contains.

Criterion	Text, every event	Filtered text (say less)	Adapter (latent)
Accuracy ($N \geq 10$)	Lowest, even calibrated	Highest	Between; above calibrated text at N = 10, 20
Prompt length	Grows with N	Content-dependent	Depends only on N
Single-request latency	Grows with N	Equal to adapter within about 5%	Flat in N
Batched throughput and energy	Does not fit at batch 8 or at $N = 50$	Between; fits only some batches	Highest; fits every configuration
Memory planning	Impossible at large N	Content-dependent	Fixed per event
False alarms	None	None	0.2 to 4.8% of routine windows, by seed and database
Auditability	Readable	Readable; normal events dropped	Needs a position-aware decoder

Against text that lists every event, the adapter wins on cost and, at moderate window sizes, on accuracy, but no careful engineer would send that prompt.

The rule is a communication policy of the kind recent edge-intelligence work formalises, where the system chooses what to send, and in what form, according to conditions [4, 19]. Existing policies choose between text and a transformer's cache. We add a third option for senders that are not transformers, and for them the choice turns on the shape of the workload and the serving regime more than on accuracy.

6.2. Why the Filter Wins on Accuracy and the Adapter Wins Under Load

The two results have different causes. The filter is accurate because it uses the sender's own labels to remove exactly the events that cannot change the answer, leaving a short list the receiver can reason over. Its accuracy therefore depends on how the sender's evidence is distributed: on the second sender, filtered text fell at 50 events when urgent windows rested on a single borderline beat among many. The adapter is less accurate for reasons this study does not separate. The receiver has to learn from a small training set to read tokens that were never part of its vocabulary, and the linear map from a classification-trained vector carries less than the sender's text fields do. The side inputs of the final adapter were added to close that second gap; the ablation shows they account for its gain, and the run-length result shows the gap only partly closed. The adapter wins under load for a structural reason: the filter's length depends on content, so batches of long prompts do not fit, whereas the adapter's length depends on N alone. The first is a property of the sender's output and the second of the interface, and a designer who knows the workload can read off which regime applies.

6.3. What the Result Means for Auditability

The standing objection to a latent channel is that no one can read it. Much can be read back by training decoders on the tokens: label, tier and abnormal-run status at the level of the input, and heart rate and interval nearly so with a linear decoder, at every slot tested. An auditor needs a position-aware decoder per slot, and the same method shows what the channel does not carry (signal quality). But recoverable is not the same as used. At one virtual token per event every field remained recoverable, even from a run that failed the task, so a recoverability audit can certify what the channel carries but not that the receiver relies on it. The run-length result shows a field that reaches the language model without being used reliably. A pre-generation score computed from the context vector offers an inspection that does not depend on the tokens at all. Zhang and Emu show, for latent channels

between language models, that equal end-task accuracy can arise from opposite mechanisms, and that a message must be replaced to know whether it matters [27]. The same distinction applies here: what the tokens hold and what the receiver does with them are separate questions, and our recoverability measurements address the first directly and the second only through task accuracy. The latent message is not a privacy measure: the recoverability results show that a trained decoder reads its content back.

6.4. What the Receiver Adds

On this task, nothing that a rule could not do: the reference answer is a rule over the sender's outputs, and the rule reaches it exactly at negligible cost (Table 5). This is a choice of design, not an oversight. Fixing the answer makes every error attributable to the channel, which is what a comparison of interfaces needs; on a task where the receiver must interpret, an error could come from the channel or from the receiver's judgement, and the two could not be separated. The study therefore says which channel to build into an agent system whose receiver is a general-purpose model, not that a language model improves heart-rhythm triage. Whether a latent channel still carries enough when the receiver must do what a rule cannot, for instance name the rhythms in a window from the timing of beats that are individually normal, against independent cardiologist annotations, is the natural next test. It is also the case where a sender-side filter should fail, because the evidence lies in the beats it removes.

6.5. Implications for System Design

Three practical points follow. First, the value of a latent channel is a property of the workload and not of the channel: report cost against the number of events and under batching, and include a filtered text baseline, or the comparison says little. Second, a fixed per-event footprint changes how a system is provisioned even where accuracy is equal, because batch size no longer depends on what the sender has to say. Third, inspectability can be measured and not merely asserted: train decoders on the message and report what they recover, per slot and per field.

6.6. Single Events and Other Settings

For a single event the latent channel offered no measurable advantage on this task: the event's content fits a short text message that the receiver reads accurately (balanced accuracy about 0.97 for text against 0.75 to 0.80 for the adapter), and replacing the event's text with four virtual tokens shortens the prompt only from 576 to 516 tokens (10%). The single-event case may still favour a latent message where these conditions do not hold. We list the settings as hypotheses, none tested here, together with one that we tested and rejected.

- **Information that the text fields omit:** The task needed only the label, the confidence and the run length, which compact text carries exactly. Questions that depend on what a structured summary drops, such as beat morphology or the shape of the classifier's uncertainty, could favour the context vector, which holds more than its label.

- **Not bandwidth:** A latent channel is sometimes expected to save transmission between devices. We measured the bytes a sensor would send for every test window and found the opposite (Table 16): the adapter's input, 35 numbers per event, is larger than the event as JSON text at full precision and, even when quantised to 8 bits, 4 to 10 times larger than compressed text or a binary encoding of the same fields at 50 events (2 to 6 times at 10 events; for a single event it is smaller than compressed text but still 3.6 times a binary encoding), because it carries the whole state of each event rather than the four fields the task needs. The channel's advantage is in the receiver's computation, not on the link, and we do not claim a bandwidth benefit.

- **Streaming into a running session:** An agent that receives each event as it occurs, appended to a cached context, pays four tokens per event instead of about sixty. Over a session this is the multi-event regime spread over time, where the advantage was observed.

Table 16: Bytes a sensor would transmit per window (median over the test windows, 8-byte header included) and time at 10 kbit/s for 50 events. Latent: the 35-dimensional per-event input of the adapter. Binary fields: label, confidence (16-bit), run length and signal quality, 5 bytes per event, rendered as text at the hub. Protocol overhead and radio energy are not modelled.

Interface	N=10	N=20	N=50	ms at 10 kbit/s, N=50
Compact JSON text	1,278	2,545	6,348	5,078
Filtered JSON text	251	252	412	330
Compact JSON, zlib	210	279	461	369
Filtered JSON, zlib	161	163	181	145
Latent, float32	1,408	2,808	7,008	5,606
Latent, float16	708	1,408	3,508	2,806
Latent, int8	362	712	1,762	1,410
Binary fields	58	108	258	206

- **Senders without a natural text form:** Our sender has an obvious text message, a class label and a confidence. Embedding models, anomaly detectors or fused multi-sensor states do not, and a text interface for them requires designing a lossy summary that a learned channel avoids.

6.7. Limitations

- **The task is a fixed rule:** The reference answer is a three-line rule over the perception outputs, so a language model is not needed for the answer. The study measures a channel, not reasoning, and says so. It does not show that the language model contributes judgement.

- **A fixed pipeline, not an autonomous agent system:** The two components form a fixed pipeline: the sender always transmits and the receiver answers one question. We use "agent" for an independently built component in such a pipeline. We do not test autonomous communication decisions, two-way exchange, tool use or multi-turn sessions, which are where an agentic system would use the channel.

- **One receiving model:** All results use Gemma 4 E4B in 4-bit quantisation. A second receiver, of a different family, is future work, and the adapter is specific to one embedding space.

- **One database for tuning, one external test:** The primary test set informed several design iterations. Each was logged before the data it affected, but the set is no longer untouched. The external confirmatory test on INCART was fixed for this reason, and it replicated the accuracy findings (Section 5.4); its first timing of text with every event listed at 50 events was distorted by memory paging and was re-measured under the memory cap, with both measurements on record.

- **Training variance and data size:** Outcomes vary between training seeds (validation balanced accuracy 0.72 to 0.82 for the final adapter, and one failed run at one token per event), so every adapter result is reported over three seeds; the eight-token setting is a single run. The adapter was trained on 447 windows. Validation curves from the twelve runs that share this training set show that most reached their best checkpoint within two passes over the data and stopped early (Figure 8), which suggests the adapter is close to capacity at this data size; a minority, including one run of the final adapter, were still improving at the three-pass limit, so a modest gain from longer training cannot be excluded. These curves concern repeated passes over a fixed set; we did not train on larger or smaller sets.

- **Perception quality:** Specificity of the encoder's rule against the annotations is 0.46 to 0.64, and fusion beats are almost never detected. The ceiling on usefulness is set upstream of the interface.

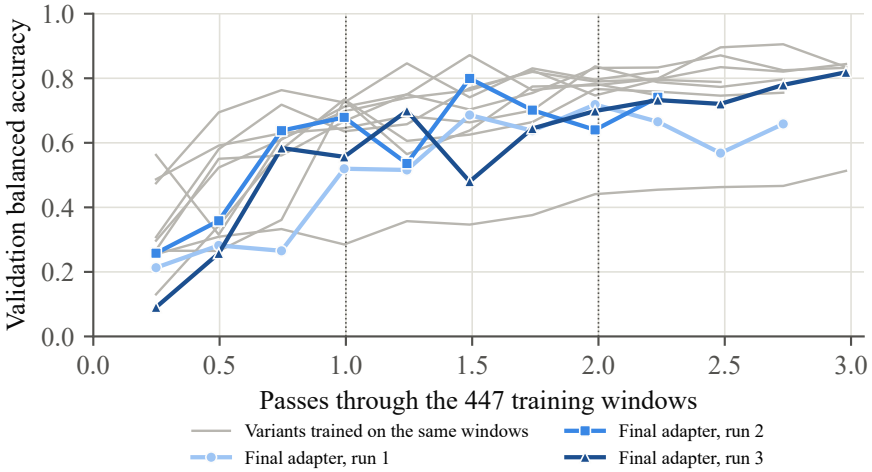


Figure 8: Validation balanced accuracy against passes through the 447 training windows, for the three runs of the final adapter and nine runs of variants trained on the same windows (exploratory; no test data). Most runs reach their best checkpoint within two passes and stop early; a few are still improving at the three-pass limit. The lowest grey curve is the one-token run that failed to learn the task.

- **Zero-shot text baseline:** The adapter was trained on the task while the text arms are zero-shot under a frozen-model constraint. A trained text baseline would narrow the accuracy gap and was not run.
- **Fairness of calibration:** Calibration chose a single bias on the routine score. A richer calibration could further improve text with every event listed.
- **Hardware and software:** Timing was measured on one consumer GPU through a general-purpose framework, because production engines do not accept continuous embeddings; only relative results are meaningful. Both agents ran on one machine: message sizes between devices were computed (Table 16), but transfer over a real link was not measured.
- **Clinical use:** The system is a research prototype. No clinician rated its outputs, and nothing here supports clinical deployment.

7. Conclusion

A perception agent that is not a transformer can talk to a frozen language model in several ways, and accuracy alone does not settle the choice. All results concern one receiving model, Gemma 4 E4B, and the conclusions are claims about that interface; whether they transfer to other receivers is untested. A sender-side filter is the more accurate and more readable interface when the receiver's question is known in advance. A linear adapter with four tokens per event gives a latent channel whose cost is fixed per event. It is more accurate than calibrated compact text at moderate window sizes, in our setup serves more decisions per second at lower median energy than the filter under batched serving on constrained hardware, and leaves each event's content recoverable from its tokens. The task fixes the answer to isolate communication fidelity, so the study says nothing about clinical reasoning or clinical value, which would need clinician evaluation or a task that requires clinical interpretation. We give a rule for choosing between the two and release the analysis plan, its change log, the harness and the analysis, so the comparison can be repeated for other senders, receivers and tasks.

Appendix A. Supplementary Methods

This appendix gives the settings needed to rebuild the system that the main text describes only in outline. The code, checkpoints and result files are in the supplementary release.

Appendix A.1. Perception Agents

Both senders were trained with the cross-entropy loss and the Adam optimiser at a learning rate of 3×10^{-4}, a batch of 256 windows and at most 30 epochs, stopping when the validation loss had not improved for five epochs. The training set was resampled with class weights proportional to inverse class frequency, with the weight of any class capped at 20 times that of the majority class; uncapped weighting (the fusion class is about one hundredth of the normal class) made validation unstable. Validation used three whole training-split recordings (109, 205 and 223), so that no validation beat shared a patient with training. The encoder used downstream was trained once with random seed 0; ten further seeds were trained to measure its variability (Section 3).

The RR branch takes five features per beat computed within a recording: the interval before the beat, the interval after it, the mean of the preceding intervals, and the ratios of the first two to that mean. A single linear layer with a rectifier maps them to 16 dimensions. In the convolutional–recurrent sender these are concatenated to the bidirectional LSTM's output at every time step before the 32-unit context LSTM. The second sender is a one-dimensional residual network: a convolution of width 7 and stride 2 with 32 channels, six residual blocks of two width-5 convolutions with batch normalisation (32, 32, 64, 64, 128 and 128 channels, with stride 2 at the third and fifth), global average pooling to 128 dimensions, concatenation with the 16-dimensional RR embedding, and a linear layer with hyperbolic-tangent activation to the 32-dimensional context vector, followed by the five-class linear head.

Appendix A.2. Prompts and Answer Schema

Every text arm and the adapter arm share one scaffold: a system prompt, a single class-neutral background paragraph that states the escalation logic without naming the current beat class, a one-sentence note about the window, the event data, and the output instructions. No demonstration examples appear in any multi-event prompt. The system prompt reads: "You are a clinical decision support assistant reviewing a single ECG beat classification. You will be given structured signal features and fixed background context for the detected beat class. Based on this information only, output a structured urgency tier and a brief justification. Do not invent information not present in the input. This is a research prototype; your output is not sanctioned clinical advice." The note reads: "The event data below contains N consecutive beats from one recording. Apply the rule to each beat and report the single most urgent tier among them." The output instructions state the rule exactly (*routine* only for a normal beat; *urgent* if either a run of three or more abnormal beats or a ventricular or fusion beat with confidence above 0.85 holds, each sufficient alone; *priority* otherwise) and ask for a JSON object with three fields: `urgency_tier`, a one-sentence `justification`, and `referenced_guideline_fact`.

The text payloads differ as follows. *A-compact* lists one JSON object per event with four fields: `label`, `confidence`, `consecutive_abnormal_beats` and `signal_quality_index`. *A-filtered* replaces the list by one object with `total_beats`, `normal_beats_not_listed` and `non_normal_beats`, a list of the same four-field objects each with its position (1 for the first beat), preceded by a sentence explaining that normal beats are given only as a count. *A-full* uses the event record of the single-event study (class, description, confidence, top three classes, four signal features, signal quality and run length). For the adapter arm the event payload is replaced by the $4N$ virtual tokens and the rest of the scaffold is unchanged. The escalation flag itself is never shown to any arm. The reference tier of an event, and of each training target, comes from that flag.

Appendix A.3. Constrained Decoding

Decoding is greedy with at most 128 new tokens. A logits processor forces the literal parts of the answer object (the key names and quotation marks, using the tokenizer's own tokenisation), restricts the tier to the three valid words and lets the model's scores choose among them, and allows the two free-text fields any token that contains no double quotation mark, backslash, brace or control character. A field closes when the model emits a token that begins the closing literal, or after 40

tokens, and cannot close while empty. After the closing brace only the end-of-sequence token is allowed. The same processor is used for every arm. For the confirmatory test the text arms stop at the tier token, because stopping there changed no tier on the primary test set.

Appendix A.4. Adapter Initialisation, Data and Selection

The projection weights are drawn from a normal distribution with standard deviation $0.02/\sqrt{35}$ and the bias is zero. The position embeddings are drawn from a normal distribution with standard deviation 0.02. The scale γ is stored as a logarithm and initialised to the median norm of the embeddings the language model produces for 4,096 vocabulary tokens drawn at random (fixed seed), so that virtual tokens start at the scale of real ones. Memory is kept near 10 GB by computing logits only at the target positions and using gradient checkpointing in the frozen model (peak 9.5 to 10.3 GB).

Training windows are drawn from the training split, excluding the three validation recordings, stratified by window tier for $N \in \{1, 5, 10, 20, 50\}$ with 25 windows per tier and N, the class of each window's most urgent beat taken in round-robin order within the cell. Hard-negative windows are routine windows whose lowest beat confidence is below 0.8, at most 15 per N, at most three per recording. Validation windows are drawn in the same way from the three held-out recordings (10 per tier and N, at most five per recording, plus up to five hard negatives per N). Cells that cannot be filled are left short, which gives the 447 training and 170 validation windows reported in Section 3. The validation score is balanced accuracy of the greedy answer (96 new tokens), computed every 111 updates, a quarter of an epoch. Early stopping needs three evaluations without improvement and cannot occur before 894 updates (two epochs), and the best checkpoint is kept. The three adapters were trained with random seeds 101, 202 and 303.

Appendix A.5. Calibration of the Text Arms

For a text arm the tier is the largest of three logits at the tier token. Calibration adds a constant to the routine logit. The constant is chosen on a grid from -4 to 8 in steps of 0.25 as the value that maximises balanced accuracy on the validation windows (ties resolved to the larger value), separately for each text arm, and is then frozen. The selected values were -3.75 for compact text and 8.0 for filtered text. The second is the upper end of the grid; calibration changed the accuracy of filtered text by about 0.01 or less.

Appendix A.6. Recoverability Decoders

For each field the decoder is trained on samples from the training split and tested on samples from the test split. Beats are drawn per class (normal, supraventricular, ventricular, fusion), 1,500 per class for training and 500 for testing, and placed at slot 0, 9, 19 or 49 of a 50-event window. For the tokens the features are the 4×2560 values of the slot, standardised and reduced to 32 principal components; the reference decoders on the 32-dimensional and 35-dimensional inputs use the standardised inputs only. The multilayer perceptron has one hidden layer of 64 units and is trained for at most 2,000 iterations; the linear decoder is a linear classifier or regressor on the same features. The classification fields are balanced accuracy for the beat label, the urgency tier and the urgent flag, and the regression fields are R^2 for heart rate and RR interval. Run status uses a separate sample of 4,358 training beats (450 with a run of three or more) and 1,888 test beats (69 with such a run).

CRediT authorship contribution statement

Alif Tasbir: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft. **Aboozar Taherkhani**: Supervision, Conceptualization, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Data availability

The ECG recordings are public: the MIT-BIH Arrhythmia Database and the St Petersburg INCART 12-lead Arrhythmia Database are available from PhysioNet [42]. The code, trained perception and adapter checkpoints, analysis plan, result files and the scripts that generate every table and figure are archived on Zenodo at `https://doi.org/10.5281/zenodo.23305537` (version v1.0-paper, MIT licence) and developed at `https://github.com/a1if/Heterogeneous-Multi-Agent-Edge-AI-for-Clinical-Decision-Support`. The Gemma 4 E4B weights are available from their publisher under the model licence.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) to help draft and edit text, write analysis and plotting scripts, and check numbers in the manuscript against the result files. The authors reviewed and edited all output and take full responsibility for the content of the published article.

References

[1] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, T. Chen, Cut the crap: An economical communication pipeline for LLM-based multi-agent systems, arXiv:2410.02506 (2024).

[2] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, J. Chen, A review on edge large language models: Design, execution, and applications, ACM Computing Surveys 57 (8) (2025). `doi:10.1145/3719664`.

[3] Y. P. Shkolnikov, Agent memory below the prompt: Persistent Q4 KV cache for multi-agent LLM inference on edge devices (2026). `arXiv:2603.04428`.

[4] O. Dib, Communication-efficient agentic edge intelligence: A survey of federated multi-agent information fusion for LLM-driven edge AI, Information Fusion 136 (2026) 104577. `doi:10.1016/j.inffus.2026.104577`.

[5] J. Dong, W. Lin, BANDMAS: Causality-inspired semantic packet scheduling for bandwidth-efficient multi-agent collaboration, arXiv:2608.00458 (2026).

[6] J. Mu, X. L. Li, N. Goodman, Learning to compress prompts with gist tokens, in: Advances in Neural Information Processing Systems, 2023.

[7] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, F. Wei, In-context autoencoder for context compression in a large language model, in: International Conference on Learning Representations, 2024.

[8] X. Cheng, X. Wang, X. Zhang, T. Ge, S.-Q. Chen, F. Wei, H. Zhang, D. Zhao, xRAG: Extreme context compression for retrieval-augmented generation with one token, in: Advances in Neural Information Processing Systems, 2024.

[9] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu, LLMLingua: Compressing prompts for accelerated inference of large language models, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2023.

[10] J. Zou, R. Qiu, G. Li, X. Yang, K. Tieu, P. Lu, K. Shen, H. Tong, Y. Choi, J. He, J. Zou, M. Wang, L. Yang, Latent collaboration in multi-agent systems, accepted at ICML 2026 (Spotlight) (2025). `arXiv:2511.20639`.

[11] T. Fu, Z. Min, H. Zhang, J. Yan, G. Dai, W. Ouyang, Y. Wang, Cache-to-cache: Direct semantic communication between large language models, in: International Conference on Learning Representations (ICLR 2026), 2026.
URL https://arxiv.org/abs/2510.03215

[12] C. Pham, B. Liu, Y. Yang, Z. Chen, T. Liu, J. Yuan, B. A. Plummer, Z. Wang, H. Yang, Let models speak ciphers: Multiagent debate through embeddings, in: International Conference on Learning Representations (ICLR 2024), 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/file/e444859b2a22df6b56af9381ad1e9480-Paper-Conference.pdf

[13] Z. Du, R. Wang, H. Bai, Z. Cao, X. Zhu, Y. Cheng, B. Zheng, W. Chen, H. Ying, Enabling agents to communicate entirely in latent space, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL 2026), 2026, pp. 27106–27129. doi:10.18653/v1/2026.acl-long.1248.

[14] Y. Zheng, Z. Zhao, Z. Li, Y. Xie, M. Gao, L. Zhang, K. Zhang, Thought communication in multiagent collaboration, in: Advances in Neural Information Processing Systems, Vol. 38, 2025, pp. 123389–123418, spotlight.
URL https://proceedings.neurips.cc/paper_files/paper/2025/file/b2b502c3629beadda06311386d2c6f73-Paper-Conference.pdf

[15] M. Rossi, P. Raghunath, E. Wu, Latent cache flow: Model-to-model communication without text (2026). arXiv:2605.22863.

[16] C. Flamant, U. Ghai, K. Shimizu, The bicameral model: Bidirectional hidden-state coupling between parallel language models (2026). arXiv:2605.11167.

[17] H. Jin, P. Kuang, Y. Yu, X. Yuan, H. Wang, Agent primitives: Reusable latent building blocks for multi-agent systems, in: Proceedings of the 43rd International Conference on Machine Learning, Vol. 306 of PMLR, 2026.
URL https://arxiv.org/abs/2602.03695

[18] Y. Liu, Beyond tokens: A unified framework for latent communication in LLM-based multi-agent systems (2026). arXiv:2606.05711.

[19] L. Dai, L. Xiang, K. Yang, A token/KV-cache communication media selection and resource allocation strategy for multi-agent collaboration, arXiv:2605.25422 (2026).

[20] X. Liu, R. Zhang, W. Yu, S. Xiong, L. He, F. Wu, H. Jung, M. Fredrikson, X. Wang, J. Gao, The vision wormhole: Latent-space communication in heterogeneous multi-agent systems (2026). arXiv:2602.15382.

[21] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, Q. Wen, Time-LLM: Time series forecasting by reprogramming large language models, in: International Conference on Learning Representations, 2024.

[22] N. Babu, J. Mathew, U. Satija, A. P. Vinod, Modality reprogramming: Adapting frozen LLMs for multi-channel EEG classification, Neurocomputing 668 (2026) 132407. doi:10.1016/j.neucom.2025.132407.

[23] Y. Zhao, J. Kang, T. Zhang, P. Han, T. Chen, ECG-Chat: A large ECG-language model for cardiac disease diagnosis, in: 2025 IEEE International Conference on Multimedia and Expo (ICME), 2025, pp. 1–6. doi:10.1109/ICME59968.2025.11209476.

[24] Y. Zhang, K. Ayush, S. Qiao, A. A. Heydari, G. Narayanswamy, M. A. Xu, A. A. Metwally, S. Xu, J. Garrison, X. Xu, T. Althoff, Y. Liu, P. Kohli, J. Zhan, M. Malhotra, S. N. Patel, C. Mascolo, X. Liu, D. McDuff, Y. Yang, SensorLM: Learning the language of wearable sensors, NeurIPS 2025 (2025). arXiv:2506.09108.

[25] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, A. Y. Ng, Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network, Nature Medicine 25 (1) (2019) 65–69.

[26] K. Singhal, S. Azizi, T. Tu, et al., Large language models encode clinical knowledge, Nature 620 (2023) 172–180.

[27] H. Zhang, M. Emu, Do latent channels actually communicate? a causal audit of latent multi-agent LLM

communication, arXiv:2607.26773 (2026).

[28] R. Lowe, J. Foerster, Y.-L. Boureau, J. Pineau, Y. Dauphin, On the pitfalls of measuring emergent communication (2019). arXiv:1903.05168.

[29] H. Liu, C. Li, Q. Wu, Y. J. Lee, Visual instruction tuning, in: Advances in Neural Information Processing Systems, 2023.

[30] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Bińkowski, R. Barreira, O. Vinyals, A. Zisserman, K. Simonyan, Flamingo: A visual language model for few-shot learning, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 23716–23736.
URL https://arxiv.org/abs/2204.14198

[31] W. Han, T. Chen, C. Duan, X. Song, Y. Yao, Y. Yang, M. A. Rosenberg, E. Liu, D. Zhao, ELF: A family of encoder-free ECG-language models (2026). arXiv:2601.18798.

[32] C. Liu, Z. Wan, C. Ouyang, A. Shah, W. Bai, R. Arcucci, Zero-shot ECG classification with multimodal learning and test-time clinical knowledge enhancement, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of PMLR, 2024, pp. 31949–31963.
URL https://proceedings.mlr.press/v235/liu24bg.html

[33] S. Wong, R. Shankar, B. Albert, G. Davis Jones, Large language models surpass domain-specific architectures for antepartum electronic fetal monitoring analysis (2025). arXiv:2509.18112.

[34] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the Association for Computational Linguistics (2024).

[35] C.-Y. Hsieh, Y.-S. Chuang, C.-L. Li, Z. Wang, L. T. Le, A. Kumar, J. Glass, A. Ratner, C.-Y. Lee, R. Krishna, T. Pfister, Found in the middle: Calibrating positional attention bias improves long context utilization, in: Findings of the Association for Computational Linguistics, 2024.

[36] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, S. Singh, Calibrate before use: Improving few-shot performance of language models, in: Proceedings of the International Conference on Machine Learning, 2021.

[37] Association for the Advancement of Medical Instrumentation [AAMI], ANSI/AAMI EC57:1998 – testing and reporting performance results of cardiac rhythm and st segment measurement algorithms (1998).

[38] Gemma Team, Gemma 4 technical report (2026). arXiv:2607.02770.

[39] A. Tasbir, A learned adapter for direct communication between a non-transformer perception agent and an LLM reasoning agent, MSc dissertation, De Montfort University, Leicester, United Kingdom (2026).

[40] G. B. Moody, R. G. Mark, The impact of the MIT-BIH arrhythmia database, IEEE Engineering in Medicine and Biology Magazine 20 (3) (2001) 45–50. doi:10.1109/51.932724.

[41] P. de Chazal, M. O'Dwyer, R. B. Reilly, Automatic classification of heartbeats using ECG morphology and heartbeat interval features, IEEE Transactions on Biomedical Engineering 51 (7) (2004) 1196–1206. doi:10.1109/TBME.2004.827359.

[42] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, H. E. Stanley, PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals, Circulation 101 (23) (2000) e215–e220.